🎓 CareerSathi (करियर साथी)

Complete Technical Architecture, Laptop Setup Manual & Comprehensive Viva/Interview Bible

GitHub Repo: github.com/sachin-saroj/college-project-career-guidance

Live URL: college-project-career-guidance.onrender.com

API Swagger Docs: college-project-career-guidance.onrender.com/api/docs

Database Engine: SQLite + Drizzle ORM (Zero-Setup Embedded DB)

🏗️ 1. Project Overview & Complete Architecture

CareerSathi ek AI-powered, full-stack career guidance and mentorship portal hai jise khas taur par underprivileged aur underrepresented students ke career gap ko bridge karne ke liye build kiya gaya hai.

Layer	Technology Stack	Core Responsibility
Frontend SPA	React 19, TypeScript, Vite 6, Tailwind CSS, Zustand, Framer Motion	Modern high-performance reactive UI, state management, client routing, PDF generation, animated transitions.
Backend API	Node.js 20+, Express.js 4.x (ES Modules), Zod, Multer, pdf-parse	RESTful API endpoints, request validation, JWT auth, resume file parsing, rate limiting, and CORS security.
AI Intelligence	Google Gemini 1.5 Flash SDK (`@google/generative-ai`)	Real-time personalized career mentorship chat, dynamic roadmap generation, ATS resume scoring.
Database & ORM	SQLite (`better-sqlite3`) + **Drizzle ORM** (`drizzle-orm` , `drizzle-kit`)	Embedded relational database, WAL mode concurrency, atomic ACID transactions, zero external server dependency.

💾 2. Database Deep-Dive: Kaha Save Hota Hai aur Kaise?

> 💡 **Key Viva Point:** Project mein koi external paid database service (MongoDB Atlas, Supabase, Neon) nahi chahiye. SQLite database file project ke andar hi run hoti hai aur Drizzle ORM use manage karta hai.

📂 Database Ka Physical Location & Lifecycle

- **Database File:** `backend/database.sqlite` (Jab project start hota hai toh yeh file automatically create ho jati hai).
- **Schema Definitions:** `backend/db/schema.js` mein `users` aur `resources` tables define hain.
- **Connection & Auto-Seed:** `backend/db/index.js` mein database initialize hoti hai, `PRAGMA journal_mode = WAL` enable hota hai, aur agar DB blank ho toh automatically default resources aur test users seed ho jate hain.
- **Unified Data Access:** `backend/db.js` function-level abstraction deta hai (`getUsers()` , `saveUsers()`) jo Drizzle ke saath atomic transactions run karta hai.

🔄 Data Save Hone Ka Exact Flow (Write Request Lifecycle)

1. Student frontend par register ya profile update karta hai (e.g., Assessment submit karta hai).
2. Request HTTP POST bankar Express route (`backend/routes.js`) par aati hai.
3. Zod middleware data validation check karta hai. Auth token verify hota hai.
4. `saveUsers()` call hota hai jo SQLite mein **Atomic Transaction** run karta hai: `db.insert(users).values(...).onConflictDoUpdate(...)`
5. Data SQLite file (`database.sqlite`) mein instant commit ho jata hai. WAL (Write-Ahead Logging) hone ki wajah se concurrent writes pe database lock ya crash nahi hota.

🖥️ Drizzle Studio: Live Database Visual GUI

Drizzle ORM ke sath **Drizzle Studio** included hai. Isse project ka live database visually browser mein table aur rows ke roop mein inspect kiya ja sakta hai:

```
# Terminal mein run karo:
npm run db:studio

# Yeh browser mein live visual GUI open kar dega:
# https://local.drizzle.studio
# Jahan tum users, assessment scores, bookmarks, passwords dekh aur live edit kar sakte ho!
```

💻 3. Dost Ke Laptop Par Setup Karne Ka Step-by-Step Guide

Agar kisi dost, examiner, ya college lab ke laptop par yeh project chalana hai, toh bas yeh 5 simple steps follow karo:

1 Prerequisites (Pehle Yeh Install Hona Chahiye)

- **Node.js**: Version 18 ya usse upar (Download: `nodejs.org`). Check karne ke liye: `node -v`
- **Git**: Version control tool (Download: `git-scm.com`). Check: `git -v`
- **VS Code**: Recommended code editor.

2 GitHub Se Clone Karo

```
# Terminal / PowerShell open karo aur run karo:
git clone https://github.com/sachin-saroj/college-project-career-guidance.git
cd college-project-career-guidance
```

3 Ek Command Mein Dependencies Install Karo

```
# Root folder se yeh chalao (yeh backend aur frontend dono install karega):
npm run install:all
```

4 Environment Variables Setup Karo (.env)

`backend` folder mein ek `.env` file banao (ya `.env.example` copy karo):

```
PORT=5000
JWT_SECRET=super_secret_jwt_key_careersathi_2026
GEMINI_API_KEY=your_gemini_api_key_here
SQLITE_DB_PATH=database.sqlite
```

Note: Agar Gemini API key nahi bhi daalte, toh assessment engine aur auth bina kisi issue ke high-precision fallback deterministic engine se chalte hain.

5 Project Run Karo (Development Mode)

Do separate terminal open karo:

```
# Terminal 1 (Backend API + SQLite):
cd backend
npm run dev

# Terminal 2 (Frontend React 19 UI):
cd frontend-v2
npm run dev
```

👉 Ab browser mein **http://localhost:5173** open karo! Pura project live run karega aur database auto-seed ho jayega!

🎯 4. Project-Specific Viva Questions (15 Deep & Practical Q&A)

Q1 CareerSathi project ka main objective aur problem statement kya hai?

Answer: India mein tier-2, tier-3 aur rural colleges ke lakho students ke paas personalized career counselling ya industry mentors ki access nahi hoti. Career counselling services bohot expensive hoti hain. **CareerSathi** ek zero-cost, accessible web platform hai jo AI aur psychometric diagnostic testing ke zariye students ko unki actual cognitive skills, personality aur interests ke hisab se exact career roadmap, free scholarships, internships aur AI resume review provide karta hai.

Q2 Project ka High-Level Architecture kaise design kiya gaya hai?

Answer: Architecture 3-Tier Layered Pattern follow karta hai:

- **Client Layer:** React 19 SPA jo client-side state (Zustand) aur routing manage karta hai.
- **Server Layer:** Express.js REST API jo JWT auth, Zod input validation, rate limiting aur Gemini AI SDK context injection handle karta hai.
- **Persistence Layer:** Embedded SQLite database jise Drizzle ORM manage karta hai with WAL (Write-Ahead Logging) mode.

Q3 Frontend ke liye React 19, TypeScript aur Vite kyu select kiya?

Answer: React 19 latest stable component architecture deta hai with superior performance. TypeScript pure codebase mein strict compile-time type safety deta hai jisse runtime bugs khatam hote hain. Vite ES Modules use karta hai, jisse cold start 300ms ke andar hota hai aur HMR (Hot Module Replacement) instant re-render karta hai jo legacy Webpack ke comparison mein 10x fast hai.

Q4 Assessment Engine (Psychometric Career Test) kaise kaam karta hai?

Answer: Assessment Engine 10 multi-dimensional diagnostic questions poochhta hai jo student ke 6 core pillars score karte hain: *Logic, Creativity, Communication, Quantitative, Leadership, Resilience*. Answers ke weighted scores calculate hote hain aur archetype identify hota hai (e.g., *The Analytical Systems Architect*). Uske basis par personalized Radar Chart, Match Score (e.g., 92%), 5-Stage step-by-step Roadmap, salary range aur market demand live generate hoti hai.

Q5 Google Gemini 1.5 Flash AI integration kaise kaam karta hai?

Answer: Backend mein `@google/generative-ai` SDK use kiya hai. Jab student AI Mentor se chat karta hai, toh backend user ka context (education, skills, assessment result, resume text) prompt ke sath inject karta hai. Is prompt engineering ki wajah se Gemini generic advice dene ke bajay specific aur personalized career advice deta hai. Agar AI API limit exceed ho ya offline ho, toh fallback deterministic engine transparently handle karta hai.

Q6 Authentication aur Authorization security kaise implement ki hai?

Answer: Stateless **JWT (JSON Web Token)** authentication use kiya hai. User jab register ya login karta hai, password `bcryptjs` se 10 salt rounds ke sath hash hokar SQLite mein store hota hai. Login hone par backend 7-day validity ka signed JWT token deta hai. Protected routes par `auth` middleware token verify karta hai, aur admin endpoints par Role-Based Access Control (RBAC) middleware check karta hai ki user ka role `admin` hai ya nahi.

Q7 Project mein Database ke liye SQLite + Drizzle ORM kyu choose kiya?

Answer: Do main reasons hain:

- **Zero Deployment Friction:** Kisi external cloud database par depend nahi rehna padta. College submission ya demo ke waqt `npm run dev` chalate hi project instantly ready ho jata hai.
- **ACID Reliability:** Drizzle ORM clean, modern type-safe schema deta hai. SQLite WAL mode mein chalta hai, jo high read/write concurrency support karta hai aur data kabhi corrupt nahi hota.

Q8 SQLite database ka data kahan save hota hai aur structure kya hai?

Answer: Data `backend/database.sqlite` file mein store hota hai. Isme do core tables hain:

- `users`: User ID, name, email, role, password_hash, education, skills, assessment scores, resume JSON, bookmarked resources.
- `resources`: ID, title, description, category, provider, skills, difficulty, url, isFree.

Drizzle ORM JSON fields ko directly serialize aur deserialize kar leta hai.

Q9 Drizzle Studio kya hai aur iska kya fayda hai?

Answer: Drizzle Studio Drizzle ORM ka built-in graphical dashboard hai. `npm run db:studio` run karne par local web interface khulta hai jahan developer ya examiner ko bina kisi third-party software (jaise DBeaver ya pgAdmin) ke pure SQLite database ke tables, records aur live edits visually dikh jate hain.

Q10 Render par deploy karne par project kaise behave karta hai?

Answer: Render par root `package.json` ka build script run hota hai jo backend aur frontend dono build karta hai. Express server production mein `frontend-v2/dist` ko static serve karta hai. Jab instance start hota hai, SQLite auto-table creation aur auto-seed check execute hota hai, jisse clean zero-downtime startup milta hai.

Q11 Resume Analyzer aur ATS Score Reviewer kaise kaam karta hai?

Answer: Student jab PDF resume upload karta hai, backend mein `multer` memory storage mein file receive karta hai aur `pdf-parse` library raw text extract karti hai. Is extracted text ko analyze kiya jata hai keywords, action verbs, quantitative metrics aur skills ke basis par aur 0 to 100 ka ATS Score aur improvement suggestions generate hoti hain.

Q12 Backend API security ke liye kya measures implement kiye hain?

Answer: 5 major security layers hain:

- **Input Validation:** Zod schema validation har payload ko sanitize aur type-check karti hai.
- **Rate Limiting:** `express-rate-limit` auth endpoints par brute-force attack rokta hai.
- **CORS Policy:** Restricted origin access taaki unauthorized domains API call na kar sakein.
- **Reverse Proxy IP Trust:** `app.set('trust proxy', 1)` for Render/Cloudflare IP identification.
- **Environment Isolation:** Secrets strictly `.env` mein rehte hain, code mein hardcode nahi hote.

Q13 Admin Dashboard ke paas kya-kya powers hain?

Answer: Admin dashboard par total registered students, completed assessments, aur active resources ke live metrics dikhte hain. Admin naye courses/scholarships add, update aur delete kar sakta hai, kisi bhi student ka role change ya account remove kar sakta hai, aur pure database ka JSON backup export ya import kar sakta hai.

Q14 Frontend mein State Management ke liye Zustand kyu choose kiya Redux ke bajay?

Answer: Redux mein bohot zyada boilerplate code (actions, reducers, dispatchers, action types) hota hai. Zustand ek modern, lightweight (1.2KB) hook-based state management library hai. Isme direct state mutators likh sakte hain aur component sirf required state subscribe karke re-render minimize karta hai.

Q15 Agar is project ko 50,000 active students ke liye scale karna ho toh kya architecture changes karoge?

Answer: Production scale ke liye: 1. Embedded SQLite ko Hosted PostgreSQL (Amazon RDS ya Supabase) mein migrate karenge with connection pooling (PgBouncer).
2. AI responses aur session caching ke liye Redis layer lagayenge.
3. Resume processing ko background message queue (BullMQ / RabbitMQ) par shift karenge.
4. Media/Resume PDFs ko S3 bucket par store karenge.

💡 5. Core Tech Stack Viva Questions (20 Fundamental Q&A)

T1 **Backend kya hota hai aur iska main responsibility kya hai?**

Answer: Backend kisi web application ka "brain" hota hai jo server-side run hota hai. Iska kaam business logic execute karna, database ke sath data read/write karna, authentication aur security verify karna, aur frontend ko REST ya GraphQL API ke zariye structured JSON data supply karna hota hai.

T2 **Frontend kya hota hai aur Single Page Application (SPA) traditional website se kaise alag hai?**

Answer: Frontend user interface (UI) hota hai jo browser mein render hota hai. Traditional multi-page applications (MPA) mein har click par server pura naya HTML page bhejta hai jisse screen reload hoti hai. SPA (React) mein ek hi HTML load hota hai aur route change hone par sirf required data fetch karke DOM dynamically update hota hai bina page reload kiye.

T3 **Database kya hota hai aur Relational (SQL) vs NoSQL mein difference kya hai?**

Answer: Database structured data storage system hai. **SQL (Relational):** Data predefined schemas ke sath tables (rows/columns) mein store hota hai with foreign keys aur ACID guarantees (e.g., PostgreSQL, SQLite, MySQL). **NoSQL:** Flexible schemas use karta hai jaise JSON documents ya key-value pairs (e.g., MongoDB, Redis).

T4 **SQLite kya hai aur yeh MySQL ya PostgreSQL se kaise alag hai?**

Answer: SQLite ek **embedded, serverless** relational database engine hai. Iske liye koi alag background process ya database server run nahi karna padta; pura database ek single disk file hota hai. MySQL aur PostgreSQL client-server databases hain jo alag server process par run hote hain aur TCP/IP port par network calls listen karte hain.

T5 **ORM (Object-Relational Mapping) kya hota hai aur iska use kyu karte hain?**

Answer: ORM ek programming technique hai jo code ke objects aur database tables ke beech mapping banata hai. Isse developers ko raw SQL strings (`SELECT * FROM users WHERE...`) likhne ke bajay programming language ki native syntax mein type-safe queries likhne ki suvidha milti hai, jisse SQL Injection attacks automatically prevent hote hain.

T6 **Drizzle ORM kya hai aur yeh Prisma se kaise better/alag hai?**

Answer: Drizzle ORM ek ultra-lightweight, zero-overhead TypeScript ORM hai jo SQL-like syntax follow karta hai. Prisma mein ek heavy Rust binary engine background mein chalta hai jo bundle size aur cold starts badha deta hai. Drizzle pure JavaScript/TypeScript mein chalta hai, serverless/edge ready hai, aur SQL queries par complete transparency aur high performance deta hai.

T7 Express.js vs FastAPI mein kya antar hai?

Answer: Express.js: Node.js (JavaScript) ka minimal, highly flexible web framework hai jo async event-loop use karta hai aur JavaScript ecosystem ke sath seamlessly integrate hota hai. **FastAPI:** Python framework hai jo type hints aur Pydantic use karta hai aur automatic Swagger generation deta hai. Data science aur Python ML models ke sath FastAPI popular hai, jabki full-stack JavaScript teams Express prefer karti hain.

T8 REST API kya hota hai aur HTTP Methods (GET, POST, PUT, DELETE) ka kya kaam hai?

Answer: REST (Representational State Transfer) ek architectural style hai jo standard HTTP methods use karke resources manipulate karta hai:

- `GET` : Data retrieve/read karne ke liye (safe & idempotent).
- `POST` : Naya resource create karne ke liye.
- `PUT / PATCH` : Existing resource ko complete ya partial update karne ke liye.
- `DELETE` : Resource ko remove karne ke liye.

T9 JWT (JSON Web Token) kya hota hai aur iske 3 parts kya hain?

Answer: JWT ek compact, URL-safe standard hai jo client aur server ke beech claims securely transfer karta hai. Yeh teen parts se milkar banta hai separated by dots (`header.payload.signature`):

- **Header:** Algorithm aur token type define karta hai (e.g., HS256).
- **Payload:** Claims jaise userId, role, expiry time store karta hai.
- **Signature:** Server ke secret key se cryptographically hash hota hai taaki token tampered na ho sake.

T10 CORS (Cross-Origin Resource Sharing) kya hota hai?

Answer: CORS ek browser security mechanism hai jo rokta hai ki ek origin (domain/port) par chalne wali web application kisi doosre origin ke resources ko unauthorized access na kare. Agar frontend `localhost:5173` par hai aur backend `localhost:5000` par, toh backend ko explicit CORS headers (e.g. `Access-Control-Allow-Origin`) return karne padte hain.

T11 Zod kya hai aur schema validation kyu critical hai?

Answer: Zod ek TypeScript-first schema declaration aur validation library hai. Jab client request bhejta hai, toh malicious ya invalid data (jaise galat email format ya chhota password) server tak pahunch sakta hai. Zod payload ko check karta hai aur invalid input par 400 Bad Request return karta hai, jisse database corrupt hone se bach jata hai.

T12 Password Hashing kya hoti hai aur Plaintext kyu store nahi karte?

Answer: Hashing ek one-way cryptographic function hai jisse original password ko unreadable hash mein convert kiya jata hai jise reverse engineer nahi kiya ja sakta. `bcryptjs` random "Salt" add karta hai aur 10 cost factor use karta hai. Isse database leak hone par bhi attacker rainbow tables ya brute force se password read nahi kar sakta.

T13 Rate Limiting kya hai aur iska kya fayda hai?

Answer: Rate Limiting kisi specific IP address se aane wali requests ki sankhya ko ek fixed time window mein restrict karti hai (e.g., 15 minute mein maximum 60 login attempts). Yeh Denial-of-Service (DoS) aur password dictionary/brute-force attacks ko block karti hai.

T14 Vite kya hai aur traditional Webpack se yeh fast kyu hai?

Answer: Webpack development ke waqt pure application ko bundle karta hai jisme bade projects par kaafi waqt lagta hai. Vite native browser **ES Modules (ESM)** use karta hai, jisse source code ko bundle karne ki zaroorat nahi padti. Sirf wahi file compile hoti hai jo browser request karta hai, jisse development server milliseconds mein start hota hai.

T15 State Management kya hai aur Zustand Redux se kyu lightweight hai?

Answer: State Management application ke shared data (jaise logged-in user profile, theme, auth token) ko globally manage karta hai taaki deeply nested components mein prop drilling na karni pade. Zustand sirf 1.2KB ka hai, bina context providers ke direct hooks deta hai, aur Redux ke boilerplate actions/reducers ko eliminate karta hai.

T16 Tailwind CSS traditional CSS se kaise different hai?

Answer: Traditional CSS mein har element ke liye custom class aur CSS file banani padti hai. Tailwind ek **utility-first CSS framework** hai jo pre-built utility classes (e.g., `flex`, `p-4`, `rounded-lg`, `text-blue-600`) provide karta hai. Isse directly JSX ke andar consistent, responsive styling fast ho jati hai aur production build mein unused CSS purge ho jati hai.

T17 Write-Ahead Logging (WAL) mode SQLite mein kya karta hai?

Answer: Traditional SQLite mein write operation ke dauran pura database lock ho jata tha, jisse readers block hote the. WAL mode (`PRAGMA journal_mode = WAL`) mein changes pehle ek separate `.wal` file mein append hote hain. Isse **readers writers ko block nahi karte aur writers readers ko block nahi karte**, jisse application concurrency dramatically improve ho jati hai.

T18 Prompt Engineering aur Context Injection kya hota hai?

Answer: LLM (Large Language Model) ko sirf user ka sawal bhejne par generic jawab milta hai. **Context Injection** mein application backend student ki profile details (skills, education, test score) ko background system prompt ke sath merge karke model ko deta hai. Isse AI ko exact pata hota hai ki woh kisse baat kar raha hai aur accurate tailored response generate karta hai.

T19 Reverse Proxy kya hota hai aur `app.set('trust proxy', 1)` kyu lagate hain?

Answer: Reverse Proxy (jaise Cloudflare, Nginx, ya Render load balancer) client aur backend server ke beech baithta hai aur SSL termination, caching aur DDoS protection karta hai. Kyunki request pehle proxy ke paas aati hai, Express ko client ka real IP nahi milta. `trust proxy` enable karne par Express proxy ke `X-Forwarded-For` header ko trust karke real client IP rate limiter ko supply karta hai.

T20 Environment Variables (.env) kyu mandatory hote hain?

Answer: Secret keys (JWT_SECRET, GEMINI_API_KEY, database passwords) ko source code mein hardcode karne se security vulnerability hoti hai, kyunki code GitHub par public ho sakta hai. Environment variables secrets ko codebase se isolate karte hain, aur alag-alag environments (development, staging, production) ke liye different configurations allow karte hain.